Neural Networks

Dr. Murari Mandal

https://murarimandal.github.io

Neural Networks

Limitations of Linear Models

- Linearity implies the *weaker* assumption of *monotonicity*, i.e., that any increase in feature cause an increase in model's output.

- Classifying images of cats and dogs:

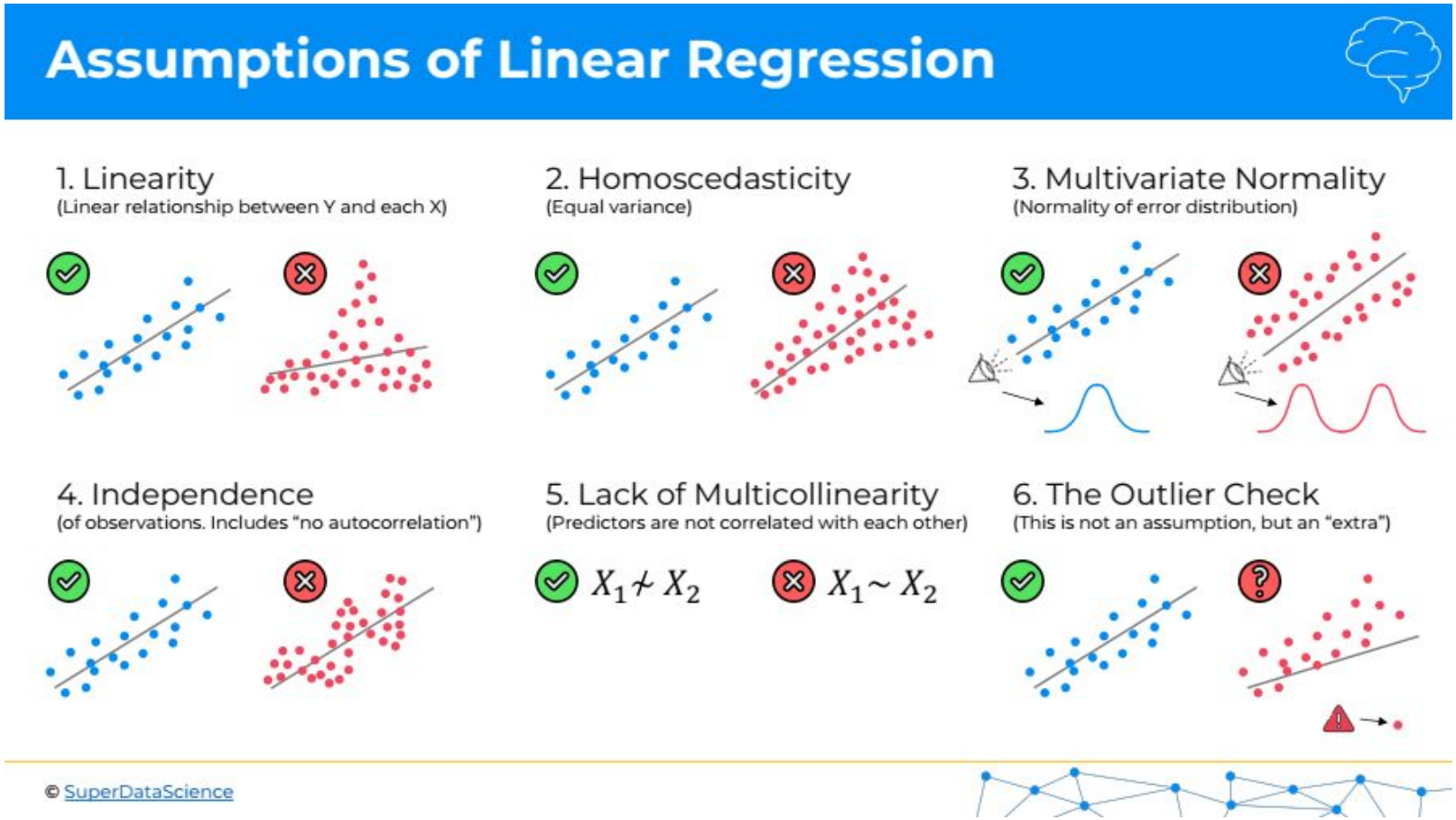
  - Increasing the intensity of the pixel at location (13, 17) always increase (or always decrease) the likelihood that the image depicts a dog?

  - Linear model -> implicit assumption that the only requirement for differentiating cats vs. dogs is to assess the brightness of individual pixels.

  - This approach is doomed to fail in a world where inverting an image preserves the category.

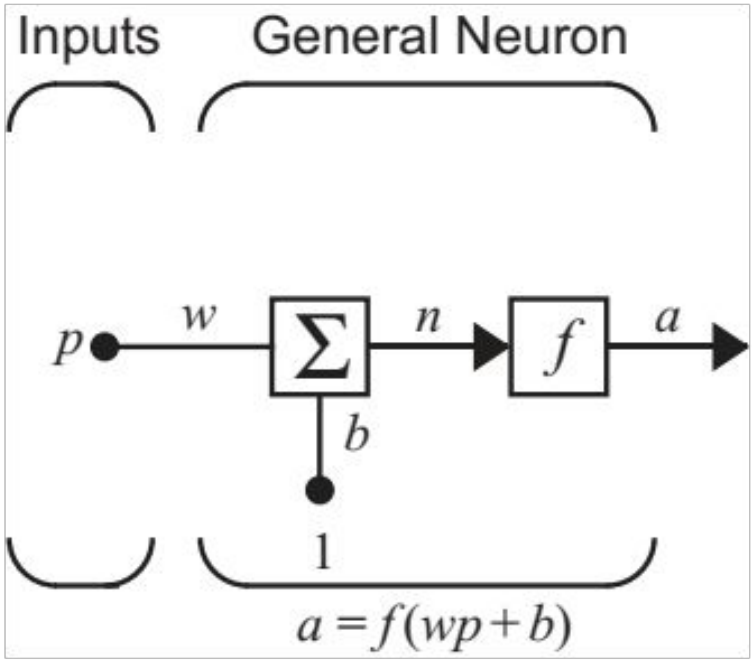
Neural Networks

Limitations of Linear Models



https://www.superdatascience.com/

Neural Networks



Single input Neuron

The neuron output is calculated as: a = f(wp + b)

If w = 3, p = 2, and b = -1.5

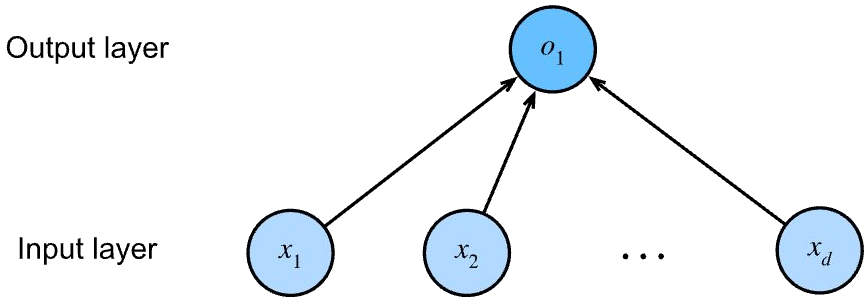
a = f(3*2 – 1.5) = f(4.5)

Hagan, Martin T., Howard B. Demuth, and Mark Beale. *Neural network design*. PWS Publishing, 1997.

Neural Networks

Activation Function

- Activation function: a) linear, b) non-linear
- Generally, neural networks use non-linear activation functions
- The selected activation function should be differentiable.



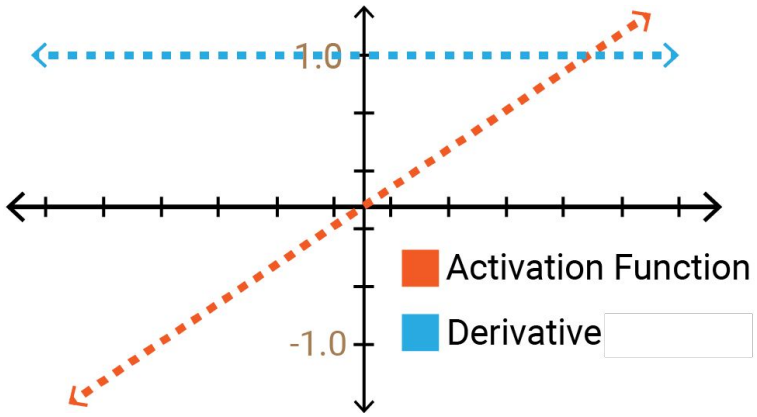
Linear regression as a single layer neural network?

What is missing? -> Activation function

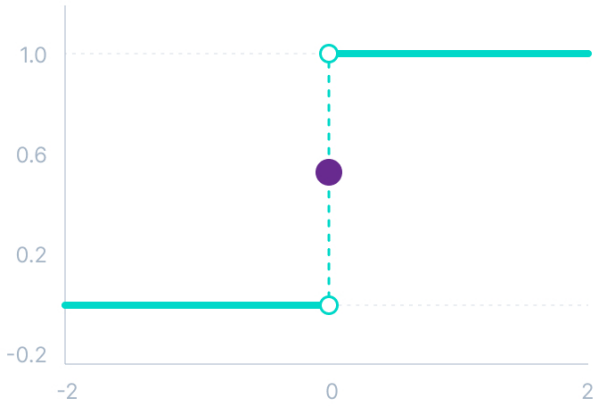
Neural Networks

Activation Function

Linear Activation function



Step Activation function



Linear

$$f(x) = x$$

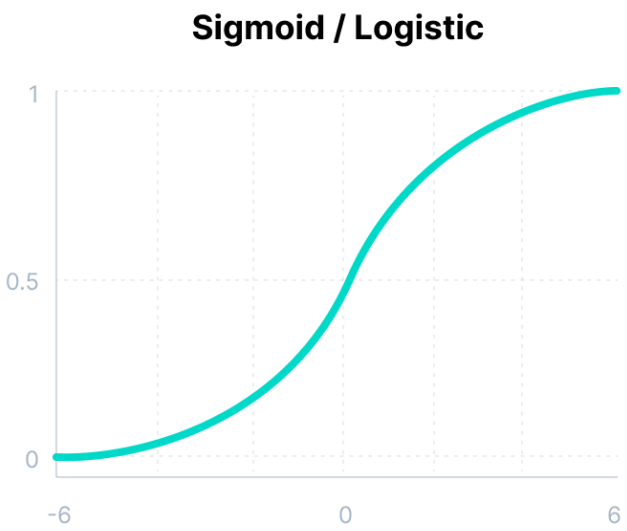
Binary step

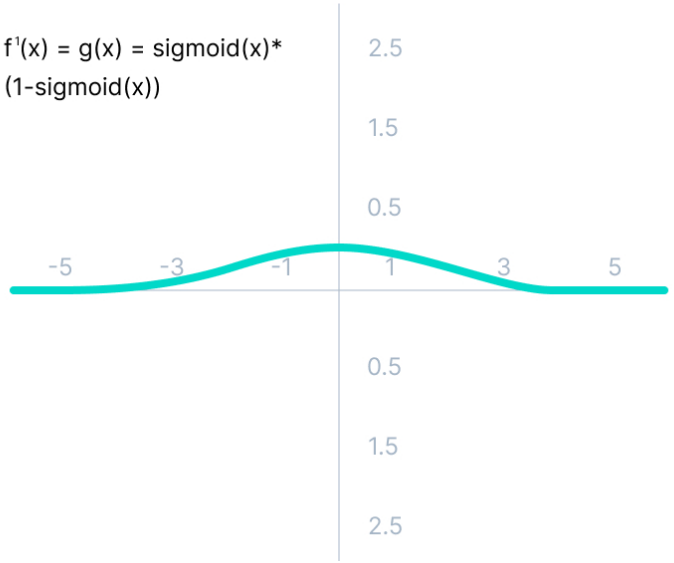
$$f(x) = \begin{cases} 0 & for\ x < 0 \\ 1 & for\ x \geqslant 0 \end{cases}$$

https://www.v7labs.com/blog/neural-networks-activation-functions

Neural Networks

Activation Function: Sigmoid





Input: Any real value
Output: Values in the range of 0 to 1

Sigmoid / Logistic

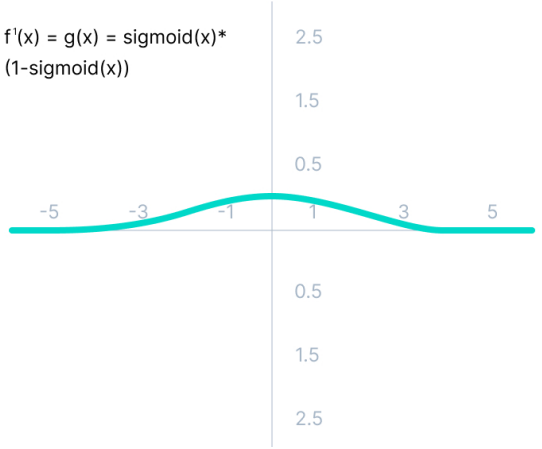
$$f(x) = \frac{1}{1 + e^{-x}}$$

Input -> More positive,
Output -> Closer to 1.0

Input -> Smaller, more negative
Output -> Closer to 0.0

https://www.v7labs.com/blog/neural-networks-activation-functions

Neural Networks



Activation Function: Sigmoid

Advantages

- It is commonly used for models where we have to predict the probability as an output.
- The function is differentiable and provides a smooth gradient, i.e., preventing jumps in output values.

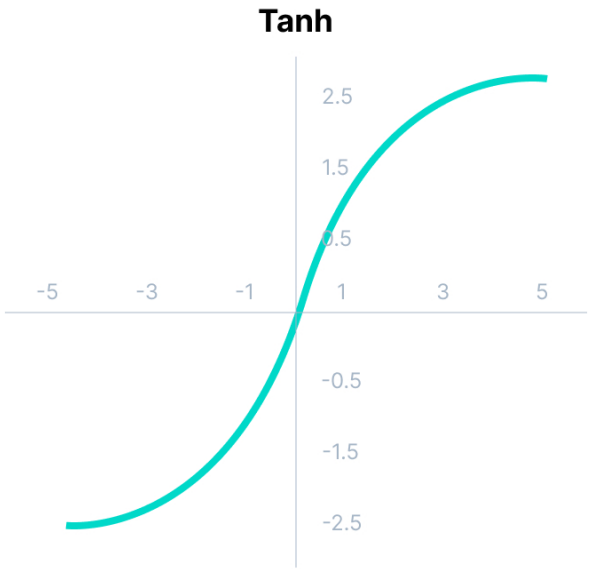
Limitations

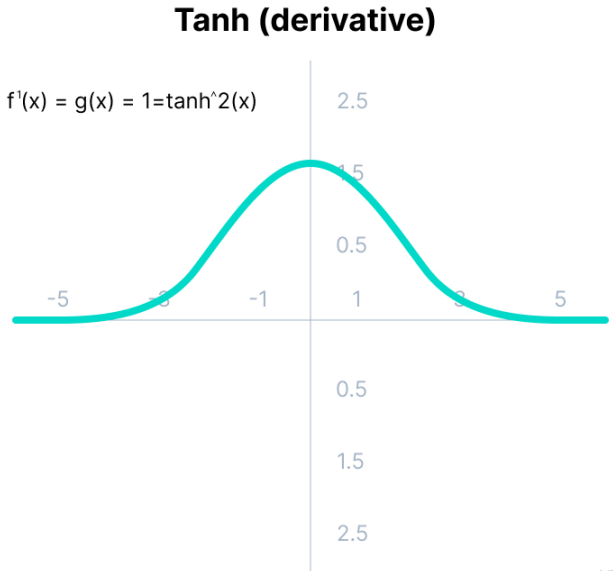
- The gradient values are only significant for range -3 to 3, and the graph gets much flatter in other regions.
- As the gradient value approaches zero, the network ceases to learn and suffers from the *Vanishing gradient* problem.
- The output of the logistic function is not symmetric around zero. So the output of all the neurons will be of the same sign. This makes the training of the neural network more difficult and unstable

https://www.v7labs.com/blog/neural-networks-activation-functions

Neural Networks

Activation Function: Tanh





Tanh

$$f(x) = \frac{\left(e^{x} - e^{-x}\right)}{\left(e^{x} + e^{-x}\right)}$$

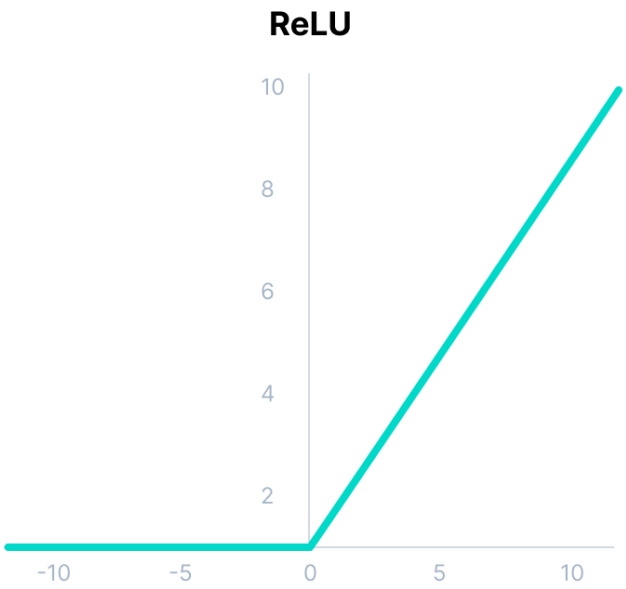
Advantages

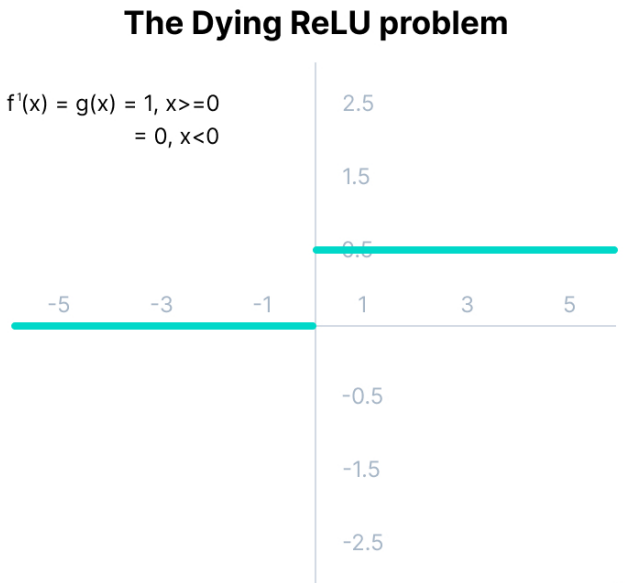
- The output of the tanh activation function is Zero centered. Hence we can easily map the output values as strongly negative, neutral, or strongly positive.
- It also faces the problem of vanishing gradients similar to the sigmoid activation function.

https://www.v7labs.com/blog/neural-networks-activation-functions

Neural Networks

Activation Function: ReLU





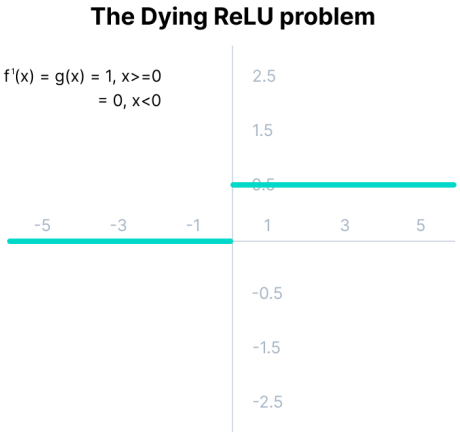
$$ReLU$$

$$f(x) = max\,(0, x)$$

https://www.v7labs.com/blog/neural-networks-activation-functions

Neural Networks

Activation Function: ReLU



Advantages:

- Since only a certain number of neurons are activated, the ReLU function is far more computationally efficient when compared to the sigmoid and tanh functions.

- ReLU accelerates the convergence of gradient descent towards the global minimum of the loss function due to its linear, non-saturating property.

Limitations:

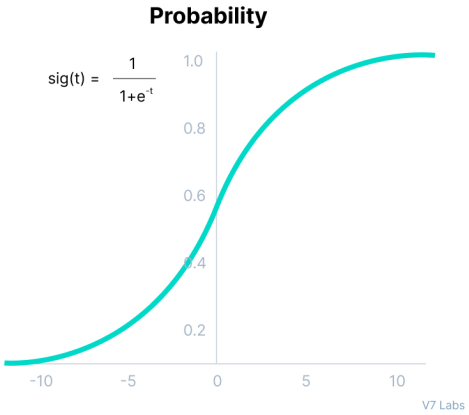
- The Dying ReLU problem

- All the negative input values become zero immediately, which decreases the model's ability to fit or train from the data properly.

https://www.v7labs.com/blog/neural-networks-activation-functions

Neural Networks

Activation Function: SoftMax



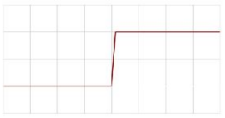
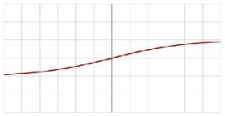
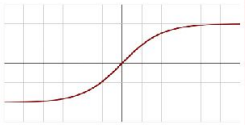
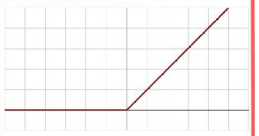
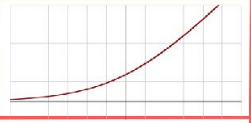
- Softmax function is a combination of multiple sigmoids.
- It calculates the relative probabilities.
- Similar to the sigmoid activation function, the SoftMax function returns the probability of each class.
- It is most commonly used as an activation function for the last layer of the neural network in the case of multi-class classification.

Softmax

$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

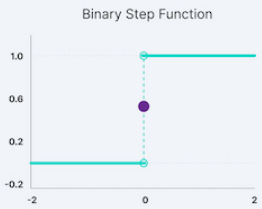
V7 Labs

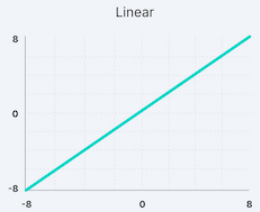
https://www.v7labs.com/blog/neural-networks-activation-functions

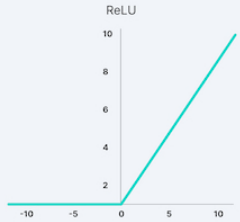
ACTIVATION FUNCTION	PLOT	EQUATION	DERIVATIVE	RANGE
Linear		f(x) = x	f'(x) = 1	(-∞, ∞)
Binary Step		$f(x) = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{if } x \neq 0 \\ \text{undefined} & \text{if } x = 0 \end{cases}$	{0, 1}
Sigmoid		$f(x) = \sigma(x) = \frac{1}{1+e^{-x}}$	$f'(x) = f(x)(1-f(x))$	(0, 1)
Hyperbolic Tangent(tanh)		$f(x) = \tanh(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$	$f'(x) = 1 - f(x)^2$	(-1, 1)
Rectified Linear Unit(ReLU)		$f(x) = \begin{cases} 0 & \text{if } x<0 \\ x & \text{if } x\geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x > 0 \\ \text{undefined} & \text{if } x = 0 \end{cases}$	[0, ∞)
Softplus		$f(x) = \ln(1+e^{x})$	$f'(x) = \frac{1}{1+e^{-x}}$	(0, 1)
Leaky ReLU		$f(x) = \begin{cases} 0.01x & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0.01 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$	(-1, 1)
Exponential Linear Unit(ELU)		$f(x) = \begin{cases} \alpha(e^{x}-1) & \text{if } x \leq 0 \\ x & \text{if } x > 0 \end{cases}$	$f'(x) = \begin{cases} \alpha e^{x} & \text{if } x < 0 \\ 1 & \text{if } x > 0 \\ 1 & \text{if } x = 0 \text{ and } \alpha = 1 \end{cases}$	[0, ∞)

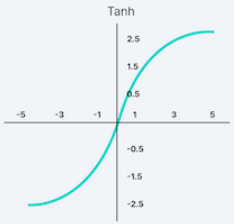
https://iq.opengenus.org/linear-activation-function/

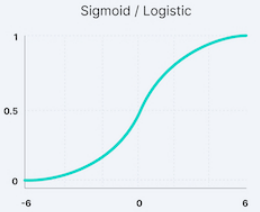
Neural Network Activation Functions

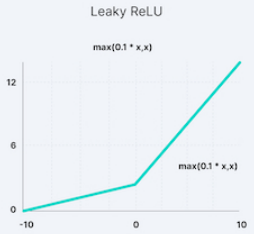


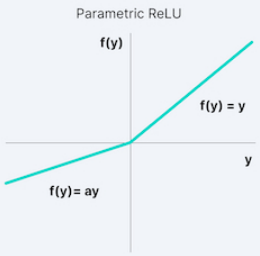


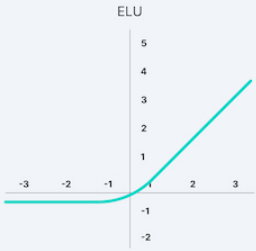


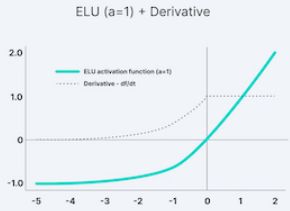


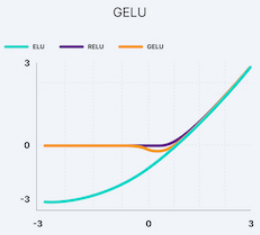


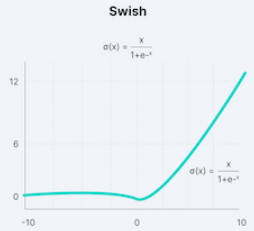








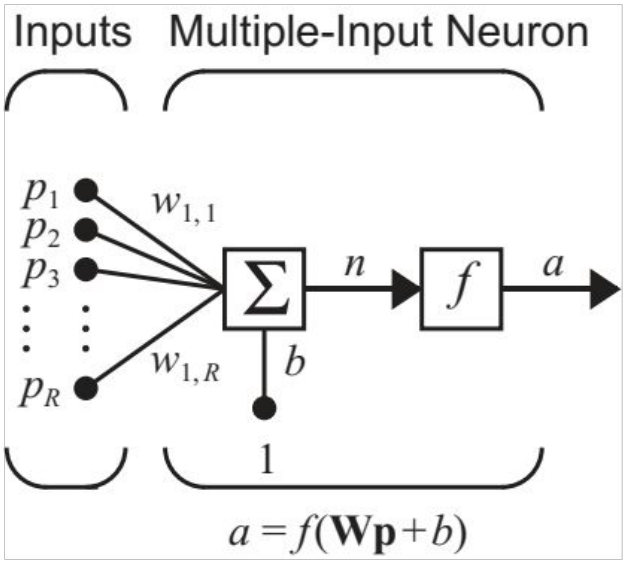


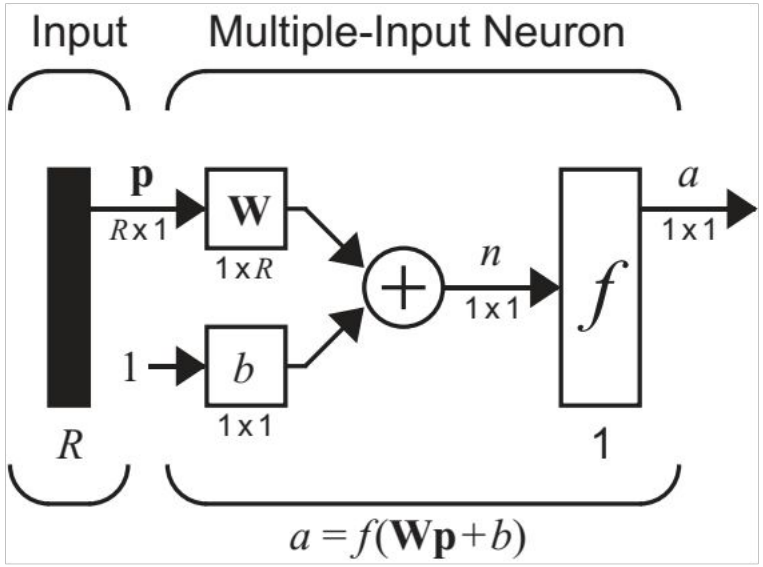


v7labs.com

https://www.v7labs.com/blog/neural-networks-activation-functions

Neural Networks





Multiple input Neuron

$$n = w_{1,1}p_1 + w_{1,2}p_2 + \cdots + w_{1,R}p_R + b$$

$$n = \mathbf{W}\mathbf{p} + b$$

$$a = f(\mathbf{W}\mathbf{p} + b)$$

p is vector of length R

W is S x R matrix

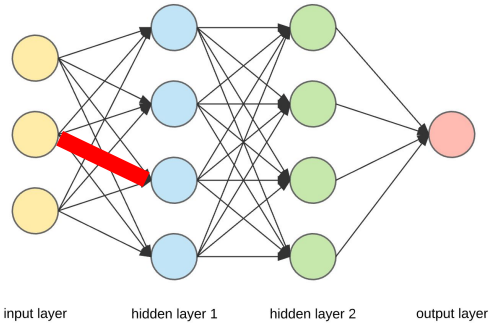
a and **b** are vectors of length S

Hagan, Martin T., Howard B. Demuth, and Mark Beale. *Neural network design*. PWS Publishing, 1997.

Neural Networks

The input vector elements enter the network through the weight matrix

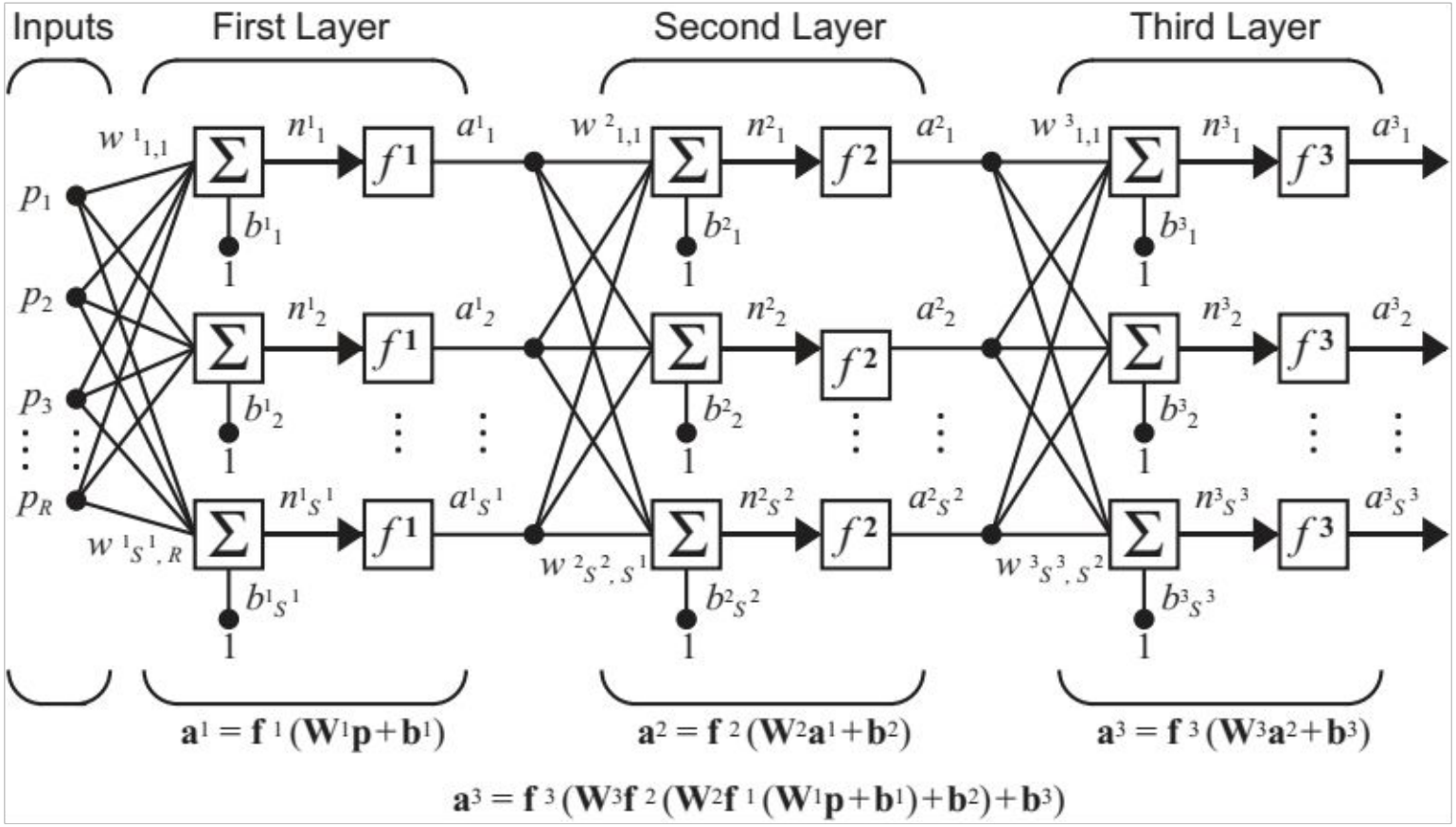
$$\mathbf{W} = \begin{bmatrix} w_{1,1} & w_{1,2} & \cdots & w_{1,R} \\ w_{2,1} & w_{2,2} & \cdots & w_{2,R} \\ \vdots & \vdots & & \vdots \\ w_{S,1} & w_{S,2} & \cdots & w_{S,R} \end{bmatrix}$$



- The row indices of the elements of matrix indicate the *destination* neuron associated with that weight
- The column indices indicate the *source* of the input for that weight.
- Thus, the indices in $\mathbf{W}_{3,2}$ say that this weight represents the connection to the *third* neuron from the *second* source.

Hagan, Martin T., Howard B. Demuth, and Mark Beale. *Neural network design*. PWS Publishing, 1997.

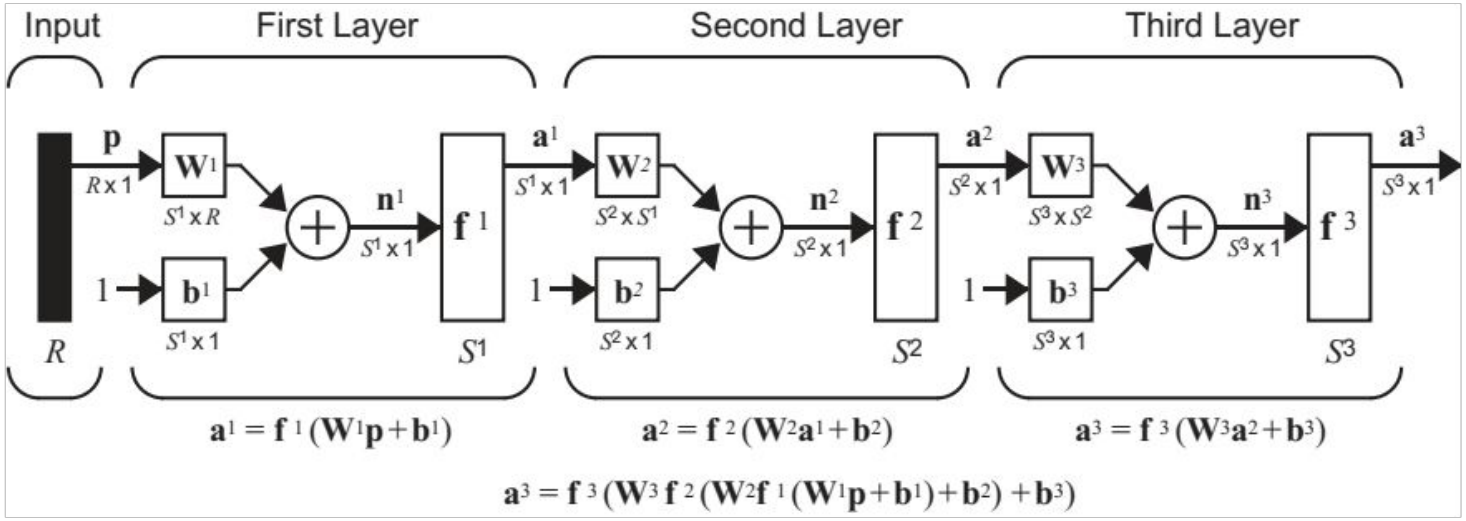
Neural Networks



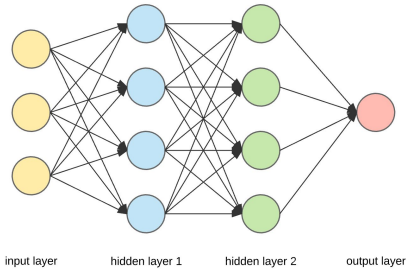
A 3 Layer Network

Hagan, Martin T., Howard B. Demuth, and Mark Beale. *Neural network design*. PWS Publishing, 1997.

Neural Networks



A 3 Layer Network, Abbreviated Notation



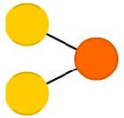
Hagan, Martin T., Howard B. Demuth, and Mark Beale. *Neural network design*. PWS Publishing, 1997.

A mostly complete chart of

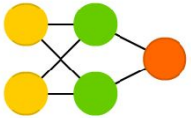
Neural Networks

- Backfed Input Cell
- Input Cell
- Noisy Input Cell
- Hidden Cell
- Probablistic Hidden Cell
- Spiking Hidden Cell
- Output Cell
- Match Input Output Cell
- Recurrent Cell
- Memory Cell
- Different Memory Cell
- Kernel
- Convolution or Pool

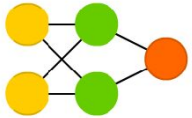
Perceptron (P)



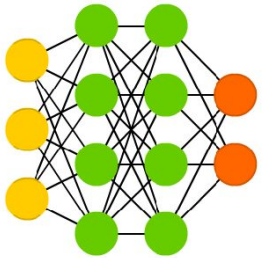
Feed Forward (FF)



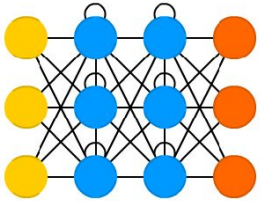
Radial Basis Network (RBF)



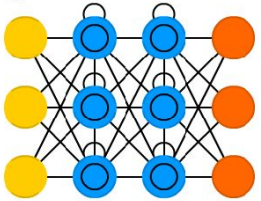
Deep Feed Forward (DFF)



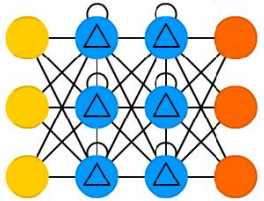
Recurrent Neural Network (RNN)



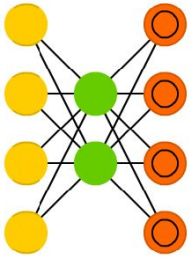
Long / Short Term Memory (LSTM)



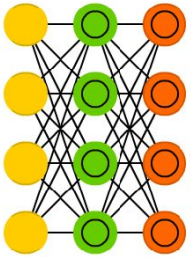
Gated Recurrent Unit (GRU)



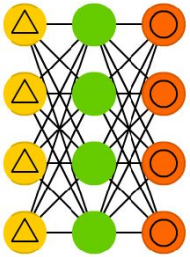
Auto Encoder (AE)



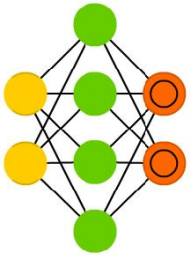
Variational AE (VAE)



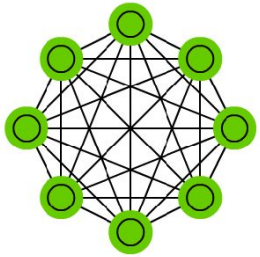
Denoising AE (DAE)



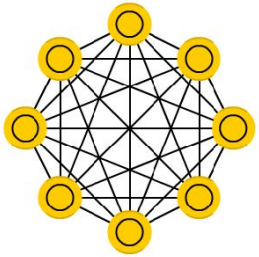
Sparse AE (SAE)



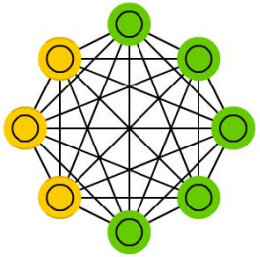
Markov Chain (MC)



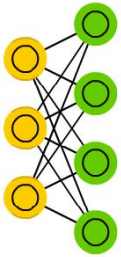
Hopfield Network (HN)



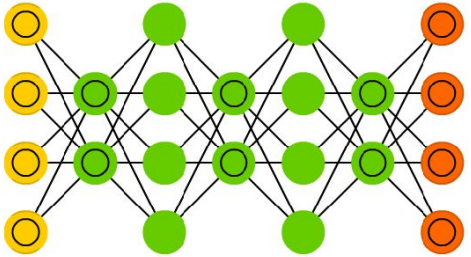
Boltzmann Machine (BM)



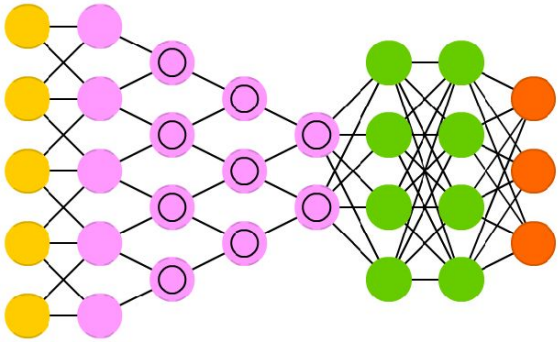
Restricted BM (RBM)



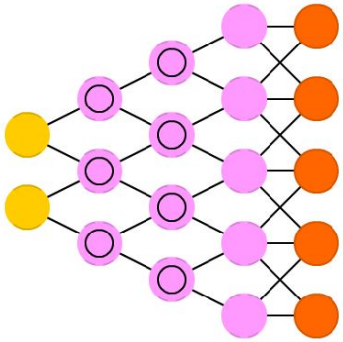
Deep Belief Network (DBN)



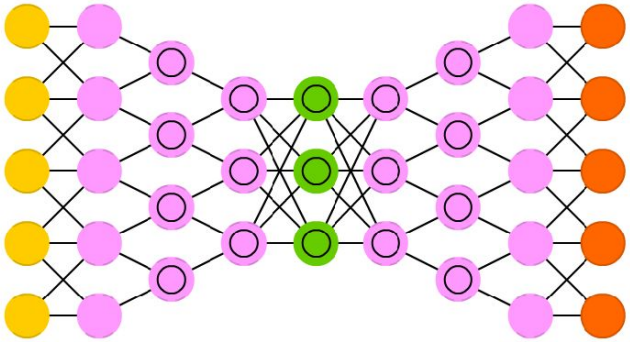
Deep Convolutional Network (DCN)



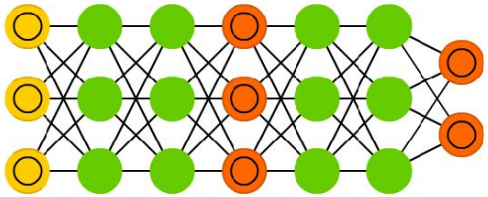
Deconvolutional Network (DN)



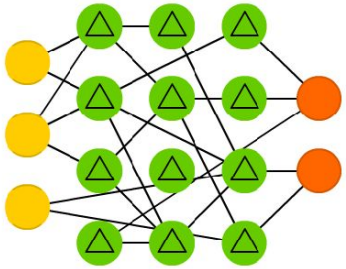
Deep Convolutional Inverse Graphics Network (DCIGN)



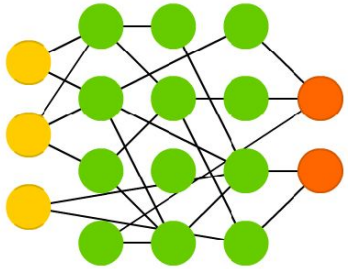
Generative Adversarial Network (GAN)



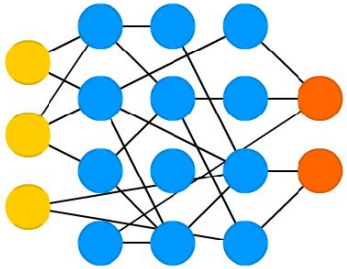
Liquid State Machine (LSM)



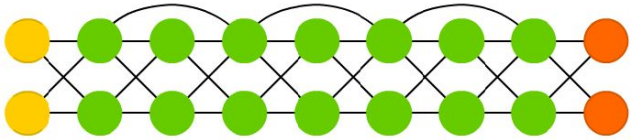
Extreme Learning Machine (ELM)



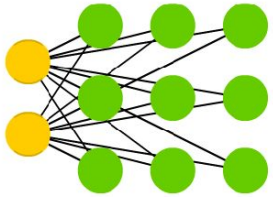
Echo State Network (ESN)



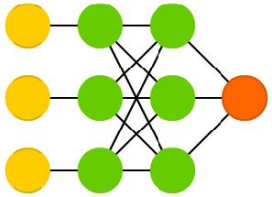
Deep Residual Network (DRN)



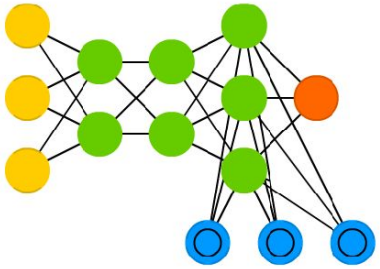
Kohonen Network (KN)



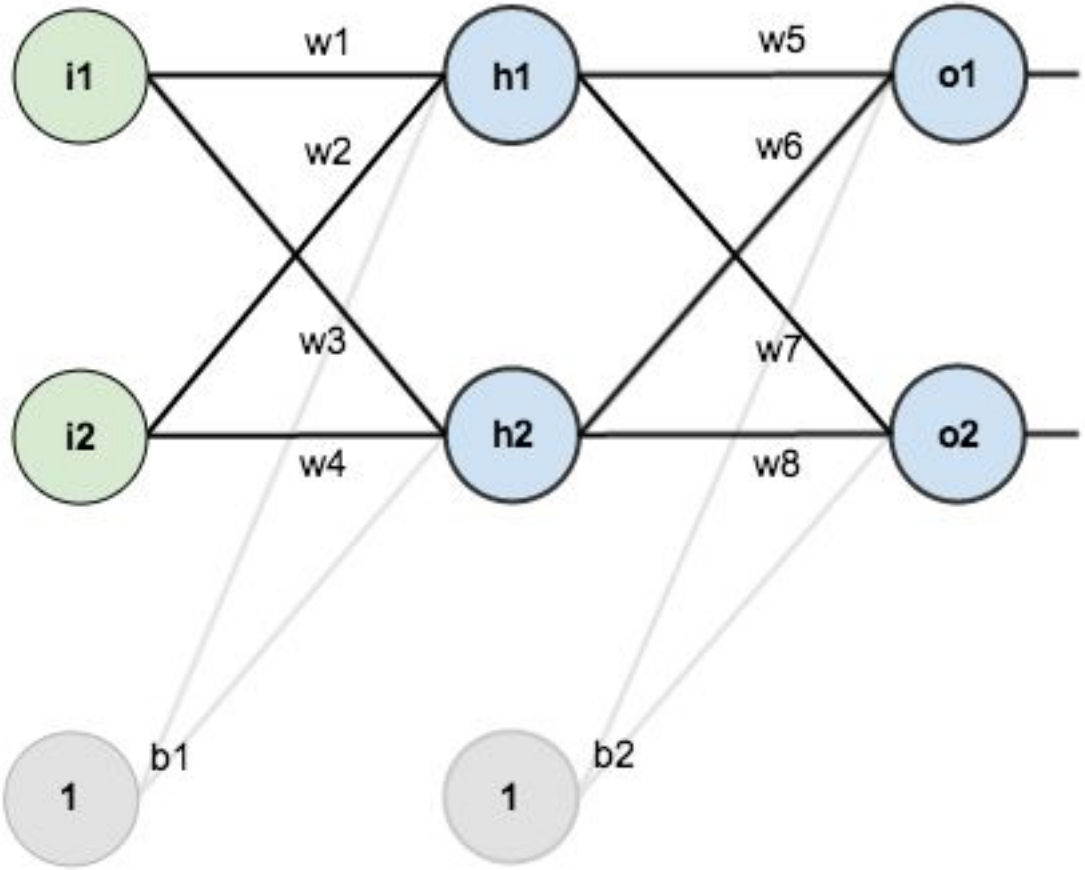
Support Vector Machine (SVM)



Neural Turing Machine (NTM)



Backpropagation in NN

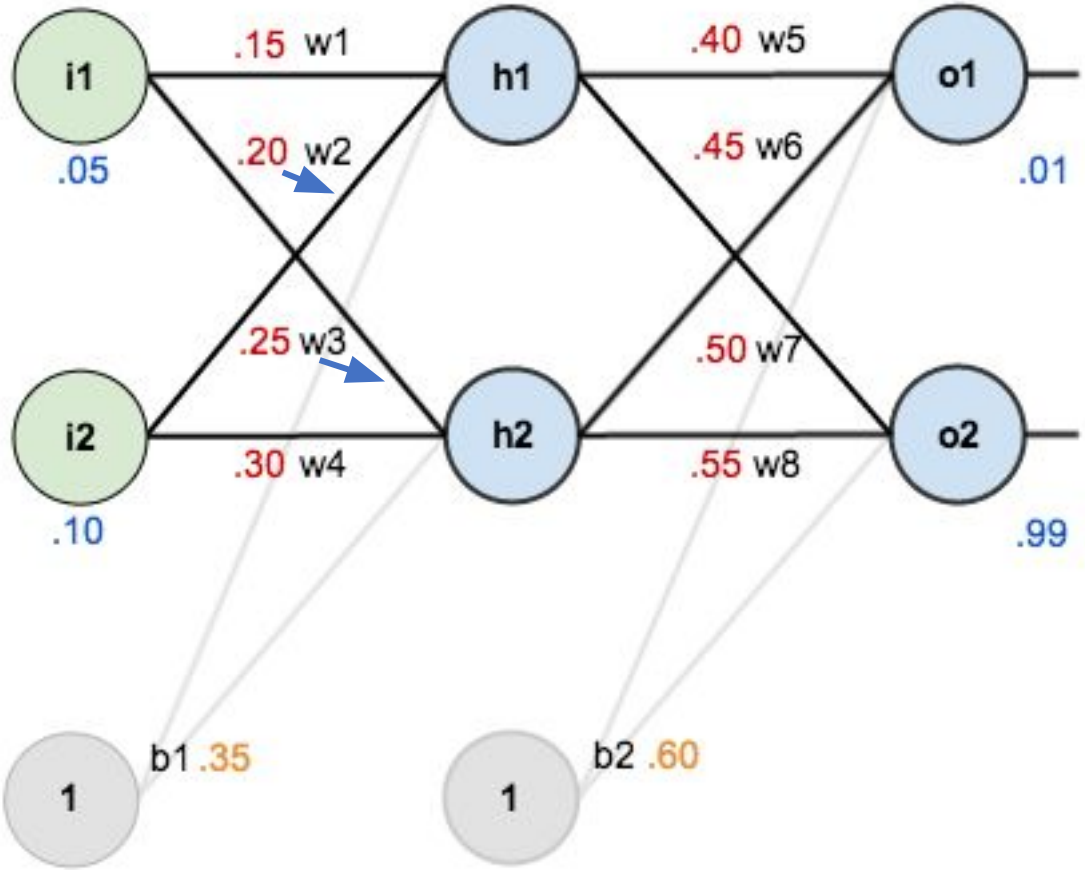


https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/

Backpropagation in NN

Input: 0.05 and 0.10

Expected output: 0.01 and 0.99



https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/

The Forward Pass

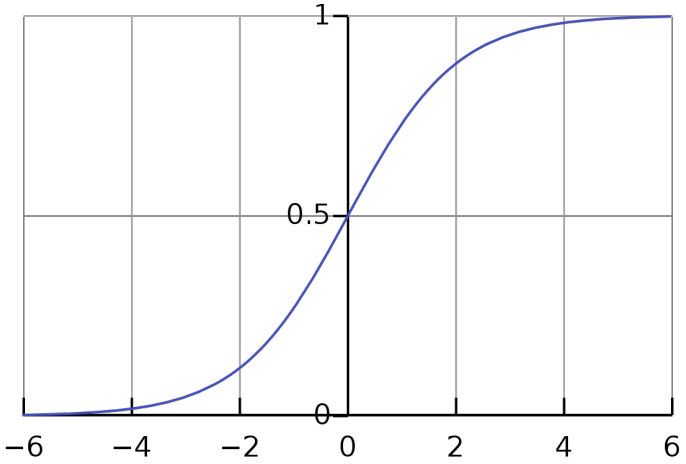
- Calculate the *total net input* to each hidden layer neuron
- *Squash* the total net input using an *activation function*
- Repeat the process with the output layer neurons.
- At the output layer, compute the Error

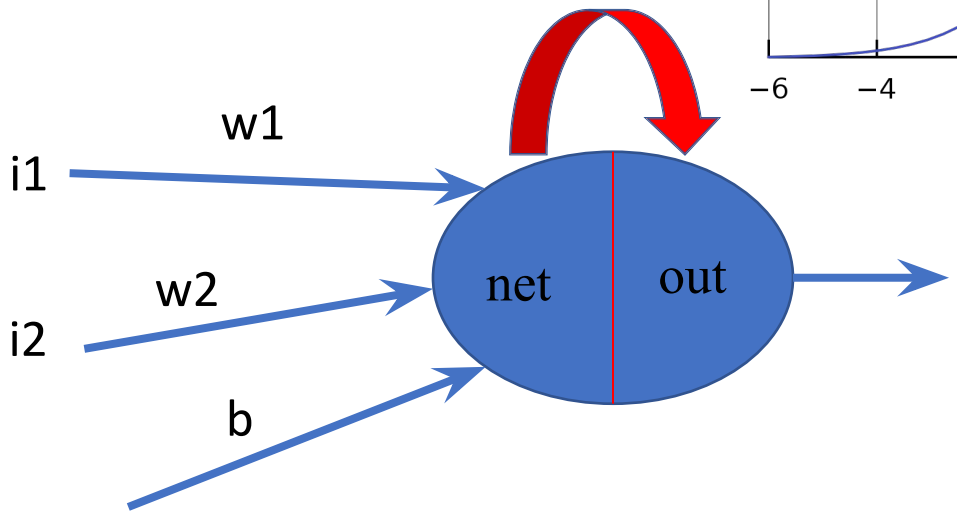
The Forward Pass

- Every Hidden node and output has 2 values:
  - Net value (net)
  - Out value (out)

$$net = w1 * i1 + w2 * i2 + b$$

$$out = \frac{1}{1 + e^{-Net}} \quad \text{(Sigmoid)}$$





The Forward Pass

net_h1= $W^{12}1 * i1 + W^{12}2 * i2 + b^{12}1 * 1$

h1 = $\frac{1}{1+e^{-Net\ h1}}$ (activation function)

net_h1 = 0.05 x 0.15 + 0.2 x 0.1 + 0.35 x 1

net_h1 = 0.3775

out_h1 = 0.5932699

out_h2 = 0.5968843

The Forward Pass

net_o1 = 0.4 x 0.5932699 + 0.45 x 0.5968843 + 0.6 x 1

net_o1 = 1.10590

out_o1 = 0.751365

out_o2 =0.772928

The Forward Pass

Calculating the Total Error/Cost

$$E = \frac{1}{2}\left\|target - o_j\right\|^2 = \frac{1}{2}\sum_j (target_j - o_j)^2$$

$$E\_o1 = \frac{1}{2}(0.01 - 0.75136507)^2$$

E_o1 = 0.274811083

E_o2 = 0.023560026

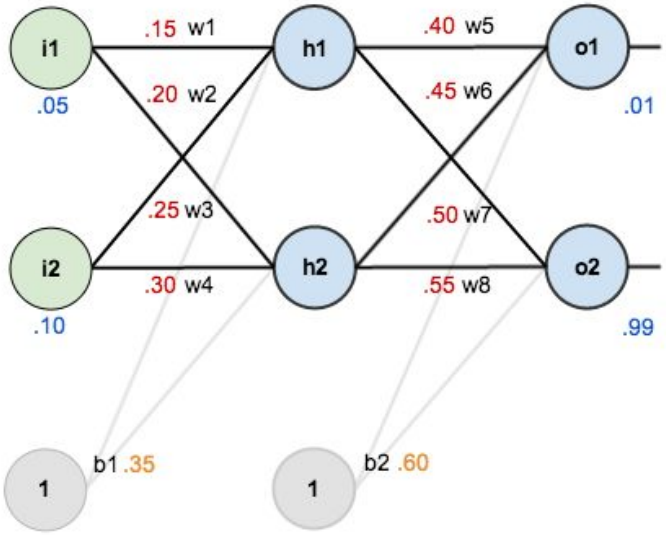
E_total = E_o1 + E_o2 = 0.298371109

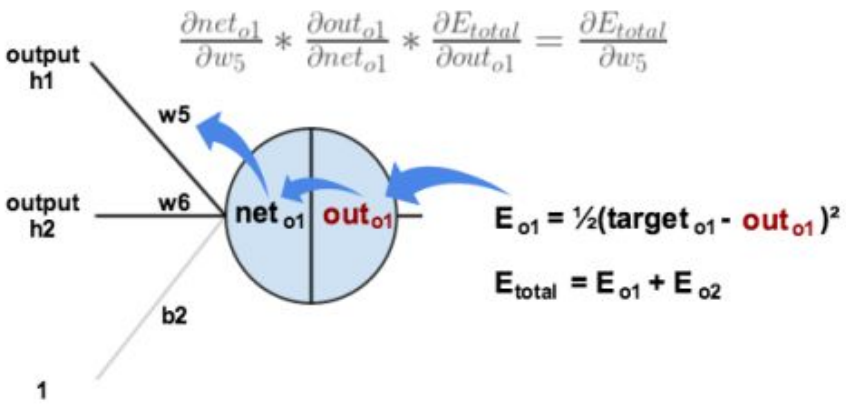
The Backward Pass

Our goal with Backpropagation:

- Update each of the *weights* in the network so that they cause the actual output to be *closer* the target output.
- Minimize the *error* for each output neuron and the network as a whole.
- For each weight ($w_1/w_2/w_3/w_4$…), we want to know how much change in w_i effects the total error i.e., $\frac{\partial Etotal}{\partial w5}$

The Backward Pass





By applying the chain rule we know that:

$$\frac{\partial Etotal}{\partial w5} = \frac{\partial Etotal}{\partial outo1} * \frac{\partial outo1}{\partial neto1} * \frac{\partial neto1}{\partial w5}$$

https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/

The Backward Pass

$$E = \frac{1}{2}\sum_j (target_j - o_j)^2$$

$$\frac{\partial Etotal}{\partial w5} = \frac{\partial Etotal}{\partial outo1} * \frac{\partial outo1}{\partial neto1} * \frac{\partial neto1}{\partial w5}$$

$$\frac{\partial E}{\partial wij} = \frac{\partial (target_j - o_j)^2}{\partial wij}$$

E is function of o_j(output)

$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial (target_j - o_j)^2}{\partial o_j} * \frac{\partial o_j}{\partial w_{ij}}$$

o_j is function of z(net)

$$\frac{\partial o_j}{\partial wij} = \frac{\partial o_j}{\partial z} * \frac{\partial z}{\partial w_{ij}}$$

$$z = w_1 x_1 + w_2 x_2 + \ldots + w_n x_n$$

The Backward Pass

E total = $\frac{1}{2}(target\_o1 - Out\_o1)^2 + \frac{1}{2}(target\_o2 - Out\_o2)^2$

1. $\frac{\partial Etotal}{\partial outo1}$ = − ($target\_o1 - Out\_o1$) + 0 = 0.74136507

   out_o1 = $\frac{1}{1+ e^{-net\ o1}}$

2. $\frac{\partial outo1}{\partial neto1}$ = out_o1 (1 – Out o1) = 0.18681560

   net_o1 = w5 x out_h1+ w6 x out_h2 + b2 x 1

3. $\frac{\partial neto1}{\partial w5}$ = Out h1 = 0.5932699

Constant are in RED color

The Backward Pass

$$\frac{\partial Etotal}{\partial w5} = \frac{\partial Etotal}{\partial outo1} * \frac{\partial outo1}{\partial neto1} * \frac{\partial neto1}{\partial w5}$$

$$\frac{\partial Etotal}{\partial w5} = 0.082167041$$

Update ***weight w5 :***

$$w5\_new = w5 - \eta \times \frac{\partial Etotal}{\partial w5}$$

W5_new = 0.40 – 0.5 x 0.082167 = .358916

η is learning rate here 0.5

The Backward Pass

w5 is now updated to *w5_new*

- In next Forward pass w5_new is used

Similarly, Find out updated values of weights w6, w7, w8 and bias b2 with the same procedure.

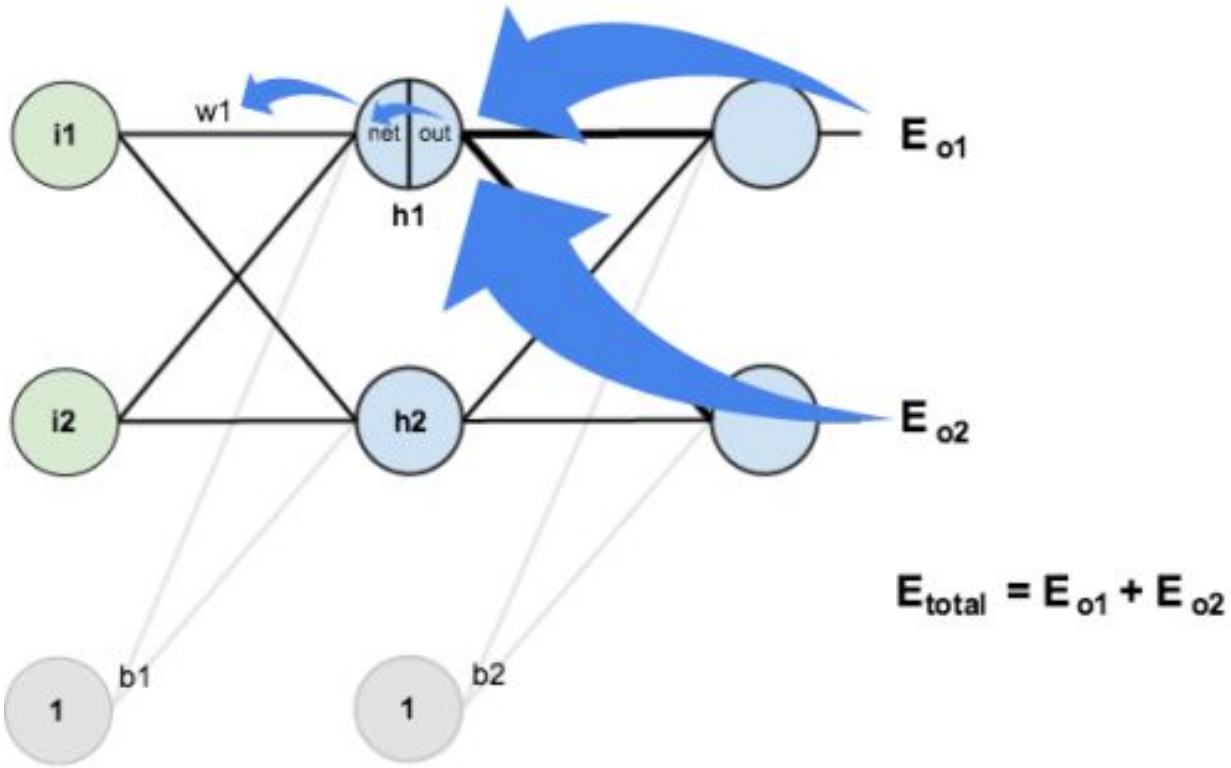
*Remember new values only considered in next Forward pass after complete update of all the weights.

The Backward Pass

For Hidden Layer :

$$\frac{\partial E_{total}}{\partial w_1} = \frac{\partial E_{total}}{\partial out_{h1}} * \frac{\partial out_{h1}}{\partial net_{h1}} * \frac{\partial net_{h1}}{\partial w_1}$$

$$\frac{\partial E_{total}}{\partial out_{h1}} = \frac{\partial E_{o1}}{\partial out_{h1}} + \frac{\partial E_{o2}}{\partial out_{h1}}$$



https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/

The Backward Pass

$$\frac{\partial Etotal}{\partial w1} = \frac{\partial Etotal}{\partial outh1} * \frac{\partial outh1}{\partial neth1} * \frac{\partial neth1}{\partial w1}$$

Etotal = Eo1 + Eo2

Eo1 = $\frac{1}{2}(targeto1 - Outo1)^2$

Eo2 = $\frac{1}{2}(targeto2 - Outo2)^2$

(Eo1 and Eo2 not directly depend on ***outh1***)

$$\frac{\partial Etotal}{\partial outh1} = \frac{\partial Eo1}{\partial outh1} + \frac{\partial Eo2}{\partial outh1}$$

we will take both separately

The Backward Pass

$$\frac{\partial Eo1}{\partial outh1} = \frac{\partial Eo1}{\partial neto1} * \frac{\partial neto1}{\partial outh1}$$

$$\frac{\partial Eo1}{\partial neto1} = \frac{\partial Eo1}{\partial outo1} * \frac{\partial outo1}{\partial neto1}$$

Both already calculated

$$\frac{\partial Eo1}{\partial neto1} = 0.74136507 \text{ x } 0.18681560 = 0.1384985$$

$$\frac{\partial neto1}{\partial outh1} = ?$$

The Backward Pass

$$\text{neto1} = w5 * outh1 + w6 * outh2 + b2 * 1$$

$$\frac{\partial neto1}{\partial outh1} = w5 = 0.40$$

$$\frac{\partial Eo1}{\partial outh1} = \frac{\partial Eo1}{\partial neto1} * \frac{\partial neto1}{\partial outh1}$$

$$= 0.1384985 \times 0.40$$

$$= 0.0553994$$

The Backward Pass

$$\frac{\partial Eo2}{\partial outh1} = \frac{\partial Eo2}{\partial neto2} * \frac{\partial neto2}{\partial outh1}$$

$$\frac{\partial Eo2}{\partial neto2} = \frac{\partial Eo2}{\partial outo2} * \frac{\partial outo2}{\partial neto2}$$

This time both not calculated

$$Eo2 = \frac{1}{2}(target\_o2 - out\_o2)^2$$

$$\frac{\partial Eo2}{\partial outo2} = -(target\ o2 - out\ o2) = -(0.99 - 0.772928)$$

$$\frac{\partial Eo2}{\partial outo2} = -0.217072$$

The Backward Pass

$$\frac{\partial Eo2}{\partial neto2} = \frac{\partial Eo2}{\partial outo2} * \frac{\partial outo2}{\partial neto2}$$

$$\text{out\_o2} = \frac{1}{1+ e^{-neto2}}$$

$$\frac{\partial outo2}{\partial neto2} = \text{out\_o2 (1 – Out o2)}$$

$$= (0.7729284)(1 - 0.7729284)$$

$$= 0.1755100$$

$$\frac{\partial Eo2}{\partial neto2} = (-0.217072) * (0.1755100)$$

$$= - 0.0380983$$

The Backward Pass

$$\frac{\partial Eo2}{\partial outh1} = \frac{\partial Eo2}{\partial neto2} * \frac{\partial neto2}{\partial outh1}$$

$$\frac{\partial Eo2}{\partial neto2} = -0.0380983$$

$$neto2 = w7 * outh1 + w8 * outh2 + b2 * 1$$

$$\frac{\partial neto2}{\partial outh1} = w7 = 0.50$$

The Backward Pass

$$\frac{\partial Eo2}{\partial outh1} = \frac{\partial Eo2}{\partial neto2} * \frac{\partial neto2}{\partial outh1}$$

$$\frac{\partial Eo2}{\partial outh1} = -0.0380983 * 0.50 = -0.0190491$$

$$\frac{\partial Etotal}{\partial w1} = \frac{\partial Etotal}{\partial outh1} * \frac{\partial outh1}{\partial neth1} * \frac{\partial neth1}{\partial w1}$$

$$\frac{\partial Etotal}{\partial outh1} = \frac{\partial Eo1}{\partial outh1} + \frac{\partial Eo2}{\partial outh1}$$

$$\frac{\partial Etotal}{\partial outh1} = 0.0553994 + -0.0190491 = 0.0363503$$

The Backward Pass

$$\frac{\partial Etotal}{\partial w1} = \frac{\partial Etotal}{\partial outh1} * \frac{\partial outh1}{\partial neth1} * \frac{\partial neth1}{\partial w1}$$

$$outh1 = \frac{1}{1+ e^{-net\ h1}}$$

$$\frac{\partial outh1}{\partial neth1} = outh1 \text{ x } (1 - outh1) = 0.5932699 \text{ X } (1- 0.5932699)$$

$$\frac{\partial outh1}{\partial neth1} = 0.2413007$$

The Backward Pass

$$\frac{\partial Etotal}{\partial w1} = \frac{\partial Etotal}{\partial outh1} * \frac{\partial outh1}{\partial neth1} * \frac{\partial neth1}{\partial w1}$$

$$neth1 = i1 * w1 + i2 * w2 + b1 * 1$$

$$\frac{\partial neth1}{\partial w1} = i1 = 0.05$$

$$\frac{\partial Etotal}{\partial w1} = 0.0363503 * 0.2413007 * 0.05$$

$$\frac{\partial Etotal}{\partial w1} = 0.00043856$$

The Backward Pass

Update ***weight w1 :***

$$w1\_new = w1 - \eta \times \frac{\partial Etotal}{\partial w1}$$

$$w1\_new = 0.15 - 0.5 * 0.00043856 = 0.149780$$

With the same procedure weights **w2 w3 w4** and bias **b1** will be computed.

w2_new = 0.19956143

w3_new = 0.24975114

w4_new = 0.29950229

The Backward Pass

- Finally, we've updated all of our weights!
- When we fed forward the 0.05 and 0.1 inputs originally, the error on the network was 0.298371109.
- After 1st round of backpropagation, the total error is now down to 0.291027924.
- It might not seem like much, but after repeating this process 10,000 times, for example, the error plummets to 0.0000351085.
- At this point, when we feed forward 0.05 and 0.1, the two outputs neurons generate:
  - 0.015912196 (vs 0.01 target) and
  - 0.984065734 (vs 0.99 target)

The Backward Pass

The Delta Rule

We can write this in short as :

$$\frac{\partial Etotal}{\partial w1} = \left(\sum_o \frac{\partial Etotal}{\partial outo} * \frac{\partial outo}{\partial neto} * \frac{\partial neto}{\partial outh1}\right) * \frac{\partial outh1}{\partial neth1} * \frac{\partial neth1}{\partial w1}$$

$$\frac{\partial Etotal}{\partial w1} = (\sum_o \boldsymbol{\delta}_o * w_{ho}) * (\text{out\_h1})\ (1 - \text{out\_h1}) * i1$$

$$\frac{\partial Etotal}{\partial w1} = \boldsymbol{\delta}_{h1} * i1$$

The Backward Pass

Summary

- We discussed a simple 3-layer neural network
- We discussed Forward Pass in a NN
- We discussed the backpropagation in NN